



API 智能测试平台

像思考一样编排，用智能来测试

AI 驱动的可视化 API 测试编排平台



开发初衷

痛点：

作为一名测试工程师和开发者，我曾尝试过市面上各种测试平台 - 从 Postman、JMeter 到 MeterSphere、Katalon，也试过自己编写自动化测试代码。但它们都存在同样的问题：

- 📄 **手工劳动太多**：为类似的场景重复编写测试代码
- ⏰ **效率太低**：创建简单的 API 测试都要花费数小时
- 🔁 **复用性差**：在不同测试用例间反复复制粘贴代码
- 💰 **学习成本高**：复杂的工具需要几周时间才能上手

解决方案：

API 智能测试平台 源于我多年的自动化测试实践经验 - 它是我自己测试框架的进化版本，专为消除重复劳动、让测试用例编写变得舒适高效而设计。

成果：

- ⚡ **快速**：几分钟内创建复杂测试流程，而非几小时
- 🎨 **可视化**：无需编码 - 只需拖拽、连接
- 🤖 **智能**：AI 理解你的意图，自动生成完整测试
- 😊 **愉悦**：终于有了一款真正好用的测试工具



快速开始



方式 1: Docker 部署（推荐）

构建 Docker（服务器部署推荐）

```
# 1. 克隆仓库
git clone https://xxx/xxx/ai-api-test.git
cd ai-api-test
```

```
# 2. 构建镜像（⚠️ 必须指定服务器 IP）
```

```
docker build -f Dockerfile.all-in-one \
  --build-arg NEXT_PUBLIC_EXECUTOR_URL=http://你的服务器IP:8001 \
  -t ai-api-test:latest .
```

3. 运行容器

```
docker run -d \
  --name ai-api-test \
  -p 3000:3000 \
  -p 8001:8001 \
  -p 8899:8899 \
  -e DATABASE_URL=file:/app/prisma/dev.db \
  -v $(pwd)/logs:/app/logs \
  --restart unless-stopped \
  ai-api-test:latest
```

💡 为什么要本地构建？

- 支持自定义 `NEXT_PUBLIC_EXECUTOR_URL`（浏览器访问执行器的地址）
- 适配不同 CPU 架构（x86_64/AMD64）
- 如果浏览器提示 `Failed to fetch`，说明需要重新构建并指定正确的服务器 IP



方式 2: 本地启动

1. 克隆仓库

```
git clone https://xxx/xxx/ai-api-test.git
cd ai-api-test
```

2. 运行初始化脚本（自动完成所有配置和构建）

```
chmod +x setup.sh
```

```
./setup.sh
```

或

```
执行 setup.bat
```

3. 启动前端（终端 1）

```
npm install
```

```
npm run dev
```

4. 启动执行器（终端 2）

```
cd executor
```

```
source venv/bin/activate
```

```
python main.py
```



默认管理员账号： `admin / admin123`

🎉 完成！ 访问 <http://localhost:3000>



什么是 API 智能测试平台？

API 智能测试平台 是新一代 API 测试平台,通过 **三大核心能力** 革新API测试 workflow:

三大核心能力



✨ 完整工作流程

1 AI 智能采集 - 零配置获取 API

访问网站 → AI 自动识别 API →
智能分类 → 生成 Schema → 保存到 API 仓库

📁 3 种采集模式:

- 浏览器插件 (Playwright) - 最简单
- HAR 文件导入 - 最快速
- mitmproxy 代理 - 最专业

2 可视化编排 - 拖拽构建测试流程

从 API 仓库选择接口 → 拖拽到画布 →
连接节点 → 配置参数 → 添加断言 → 保存用例

🎨 支持复杂场景:

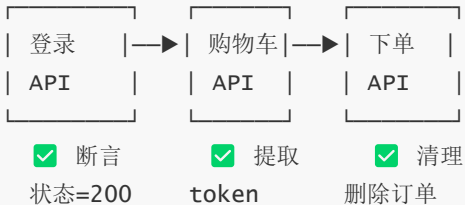
- 变量引用和传递
- 并行执行
- 断言和等待

3 AI 生成用例 - 自然语言秒变测试

描述需求: "测试用户登录后创建订单"



[AI 智能思考: 搜索 API + 设计流程 + 生成断言]



📁 自动统一分类: 所有生成的用例归属同一业务分类

🏷️ 智能标签: 自动标记测试类型和场景

🌟 API 智能测试平台优势

🚀 测试创建速度提升 10 倍

功能特性	API 智能测试平台	Postman	JMeter	MeterSphere	Katalon
🤖 AI 智能生成	✅ AI 驱动	❌ 手动编写	❌ 手动编写	❌ 手动编写	⚠️ 有限支持
🎨 可视化流程	✅ 拖拽式	⚠️ 有限	❌ XML/代码	⚠️ 有限	✅ 支持
📺 实时监控	✅ 实时仪表盘	⚠️ 执行后	✅ 支持	✅ 支持	⚠️ 有限
📁 智能捕获	✅ 3 种模式	⚠️ 仅浏览器	❌ 手动	⚠️ 仅 HAR	⚠️ 基础
📈 学习曲线	● 1小时	● 4 小时	● 2 天	● 1 天	● 1 天
🌐 多语言	✅ 中英文	✅ 多语言	⚠️ 有限	✅ 中英文	✅ 多语言
💰 成本	● 免费开源	● 部分免费	● 免费	● 部分免费	● 付费

🎯 核心优势

🏆 三大杀手级功能

功能	传统方式	API 智能测试平台	效率提升
📁 API 采集	手动记录 URL 和参数	AI 自动识别和分类	10x
🎨 流程编排	编写脚本代码	拖拽可视化构建	20x
🤖 用例生成	逐个手写测试用例	自然语言一键生成	50x

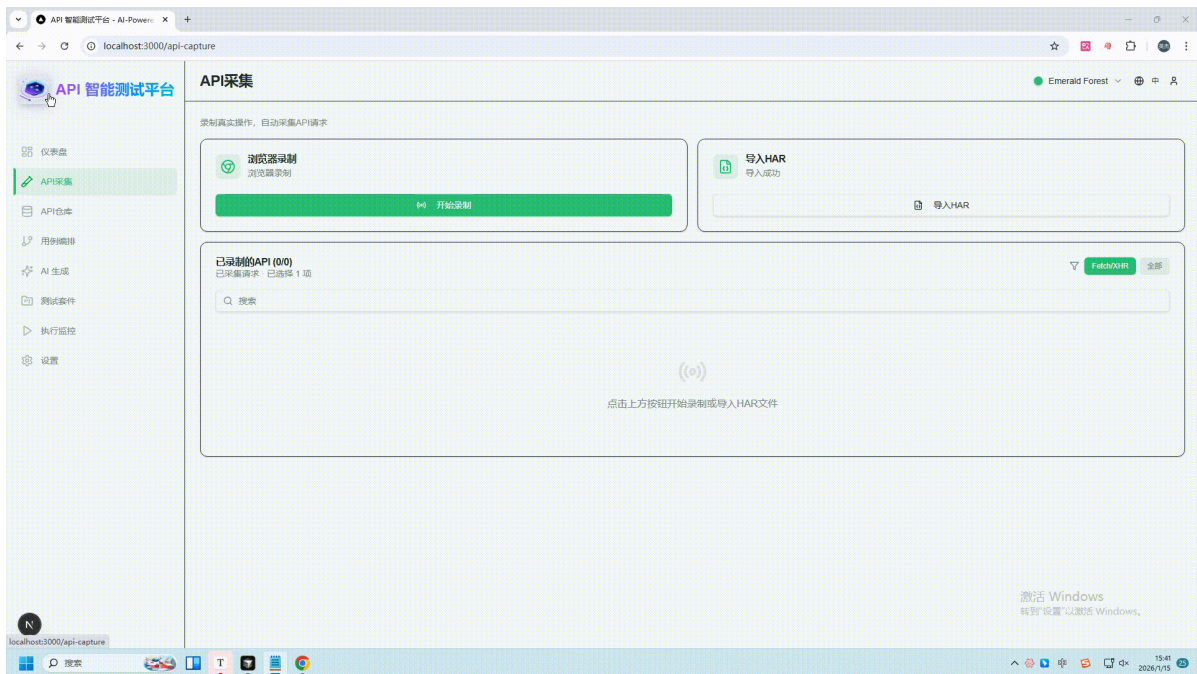
🌟 更多特性

- ⚡ **实时反馈**: 实时查看测试执行和详细日志
- 📁 **智能分类**: 自动统一管理测试用例分类
- 🌐 **多语言**: 内置中英文支持
- 🎨 **主题定制**: 支持明暗主题切换, 可自定义配色方案
- 🔧 **可扩展**: 插件系统和自定义函数
- 🚢 **易部署**: Docker、Windows、Linux、macOS

🌟 核心功能详解

完整 workflow: 📁 采集 API → 🎨 编排流程 → 🤖 AI 生成 → ✅ 执行测试

1 AI 智能采集 - 零配置获取 API



三种采集模式，适应不同场景：

🌐 模式 1: Playwright 浏览器插件（推荐新手）

访问前端页面进行采集

访问 <http://localhost:3000/api-capture>

选择 "Playwright 浏览器录制" → 输入目标 URL → 点击 "开始录制"

- **✅ 最简单:** 无需任何配置
- **✅ 可视化:** 实时看到采集的 API
- **✅ 智能过滤:** 自动过滤静态资源
- **✅ 分类识别:** AI 自动识别 API 业务分类

📦 模式 2: HAR 文件导入（最快速）

导出 HAR 文件 → 拖拽上传 → 智能解析 → 保存到仓库

- ⚡ **秒级导入:** 支持批量导入
- 🎯 **精准提取:** 请求/响应完整保留
- 🗂️ **智能分组:** 自动按域名和路径分类

🔒 模式 3: mitmproxy 代理（最专业）

适用场景:

- 📱 移动应用 (iOS/Android APP)
- 💬 小程序 (微信、支付宝)
- 🌐 浏览器 HTTPS 流量
- 💻 桌面应用程序

快速启动:

1. 在 web 界面启动

访问 <http://localhost:3000/api-capture>
选择 "mitmproxy 代理录制" → 点击 "启动录制"

2. 配置系统代理

macOS: 系统设置 → 网络 → 代理 → HTTP/HTTPS → localhost:8899
Windows: 设置 → 网络 → 代理 → 手动 → localhost:8899

3. 安装 HTTPS 证书 (首次使用)

访问 <http://mitm.it> → 下载对应平台证书 → 安装并信任

4. 开始捕获

访问任何网站或启动 APP → 实时查看捕获的请求

🌐 独立 Chrome 浏览器代理 (推荐) :

不影响现有浏览器, 启动一个独立的 Chrome 实例挂载代理:

Windows (CMD)

```
start chrome --proxy-server="你的服务器IP:8899" --user-data-dir="C:\Temp\ChromeProxyProfile"
```

Windows (PowerShell)

```
& "C:\Program Files\Google\Chrome\Application\chrome.exe" --proxy-server="你的服务器IP:8899" --user-data-dir="C:\Temp\ChromeProxyProfile"
```

macOS

```
/Applications/Google\ Chrome.app/Contents/MacOS/Google\ Chrome --proxy-server="你的服务器IP:8899" --user-data-dir="/tmp/ChromeProxyProfile"
```

Linux

```
google-chrome --proxy-server="你的服务器IP:8899" --user-data-dir="/tmp/ChromeProxyProfile"
```

💡 `--user-data-dir` 参数创建独立的浏览器配置, 不会加载主浏览器的书签、扩展和登录状态, 实现彻底隔离。

核心特性:

- 🔒 **HTTPS 完整支持:** 自动解密 HTTPS 流量
- 📺 **实时推送:** SSE 技术实时显示捕获的请求
- 🎯 **精确控制:** 启动、暂停、继续、停止、清空
- 📁 **完整数据:** 请求/响应头、参数、Body 全记录
- 💾 **批量保存:** 选中请求批量保存到 API 仓库

📖 详细文档: [mitmproxy 完整使用指南](#) →

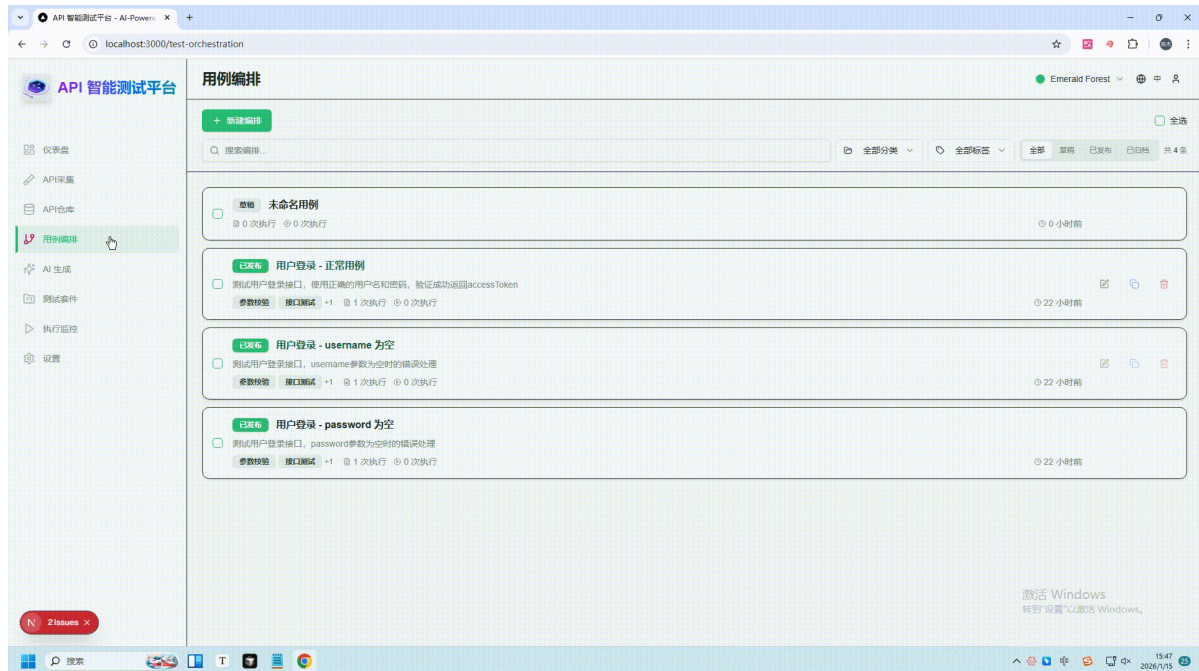
采集后自动完成:

- 📄 生成完整的 API Schema (参数、类型、示例值)
- 🏷️ 智能分类标签

-  保存到 API 仓库，随时可用

 [详细教程 →](#)

2 可视化编排 - 拖拽构建测试流程



从 API 仓库到测试用例，只需拖拽：

5 分钟构建复杂测试场景

1. 从左侧 API 仓库拖拽接口到画布
2. 连接节点，配置参数
3. 添加断言和变量提取
4. 保存，立即可执行

强大的编排能力：

-  **多种节点**: API、并发、等待、断言
-  **智能引用**: 拖拽即可引用前置节点的响应数据
-  **丰富断言**: 状态码、JSONPath、响应时间、正则表达式
-  **流程控制**: 串行、并行执行
-  **实时保存**: 自动保存，支持撤销/重做

节点类型：

-  **API 节点**: 发送 HTTP 请求
-  **并发节点**: 同时执行多个 API
-  **等待节点**: 延迟执行
-  **断言节点**: 验证响应数据

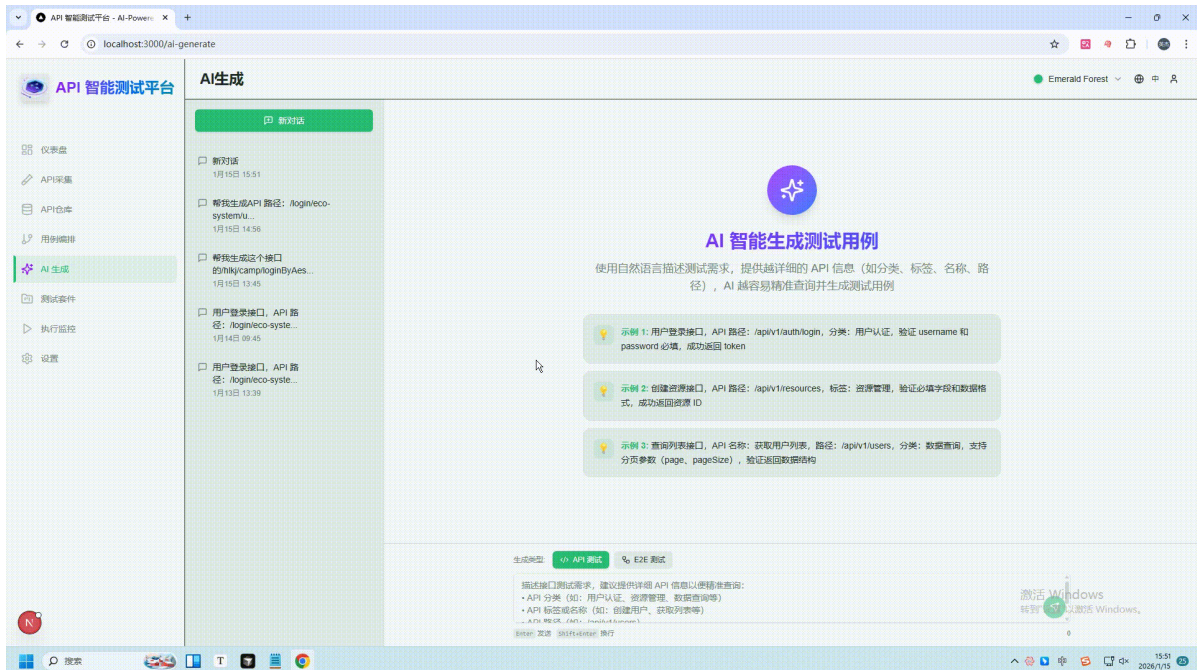
变量系统：

-  **响应提取**: JSONPath、正则、全量提取
-  **变量引用**: `{{变量名}}` 语法

-  **变量传递**: 跨节点自动传递
-  **运行时函数**: `\${random(8)}`、`\${timestamp()}`

 [详细教程](#) →

3 AI 生成用例 - 自然语言秒变测试



只需简单描述，AI 自动生成完整测试用例：

示例 1: 接口测试

输入："测试用户创建接口，username 和 email 不能为空"

AI 自动生成：

- ✓ 用户创建 - 正常用例（含后置清理）
- ✓ 用户创建 - username 为空
- ✓ 用户创建 - email 为空

📁 统一分类：用户管理

🏷️ 标签：["参数校验", "接口测试", "AI生成"]

示例 2: E2E 流程测试

输入："测试用户登录后创建订单的完整流程"

AI 自动生成：

- ✓ E2E - 用户登录 - 创建订单（3个步骤，含清理）
- ✓ E2E - 用户登录 - 查询订单（2个步骤）
- ✓ E2E - 异常：未登录创建订单（1个步骤）

📁 统一分类：订单管理

🏷️ 标签：["E2E测试", "流程测试", "AI生成"]

AI 自动完成：

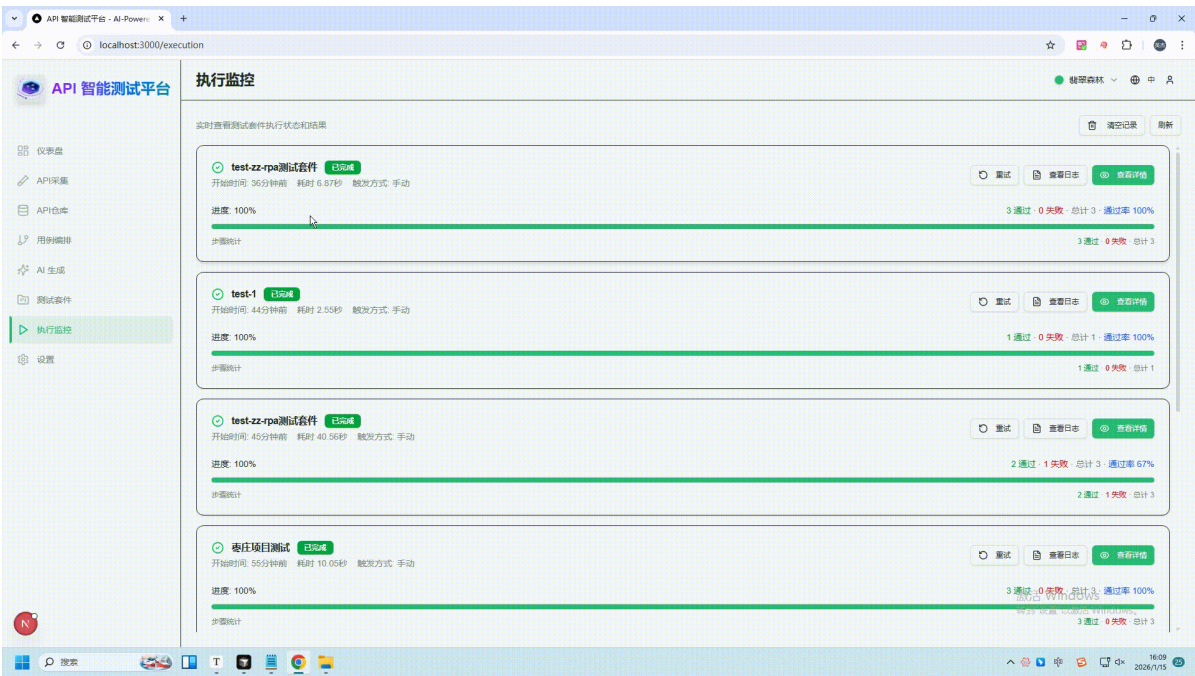
- 🔍 **智能搜索**: 从 API 仓库搜索相关接口
- 🎯 **场景设计**: 生成正常 + 异常用例
- 📁 **统一分类**: 同批生成的用例使用相同分类
- ✅ **智能断言**: 自动添加合理的断言
- 🔗 **变量引用**: 自动识别和配置变量传递
- ✂️ **后置清理**: 为成功用例添加清理步骤
- 💾 **直接保存**: 一键保存到数据库

支持的 AI 提供商:

- OpenAI (GPT-4, GPT-3.5)
- DeepSeek (深度求索) ⚡ 性价比首选
- Claude (Anthropic)
- 百度文心一言
- 阿里通义千问
- 智谱 AI
- Ollama (本地部署, 完全免费)

[📖 详细教程 →](#)

4 📁 实时执行监控 - 全链路可观测



实时查看测试执行过程:

- 📊 **实时仪表盘**: 通过 SSE 实时状态更新
- 🎯 **节点级日志**: 点击任意节点查看详细执行信息
- ⏸️ **暂停/继续**: 控制执行流程
- ↩️ **失败重试**: 一键重试失败的测试
- 📄 **详细报告**: 请求/响应、耗时、断言结果

-  **执行历史:** 追踪趋势变化

技术架构

技术栈

前端:

Next.js 16 + React 19 + TypeScript

- └─ UI 组件: shadcn/ui + Tailwind CSS 4.0
- └─ 主题系统: 支持明暗主题切换, 可自定义配色
- └─ 流程编辑器: React Flow
- └─ 状态管理: React Query + Context
- └─ 国际化: next-intl

后端:

Node.js + Python

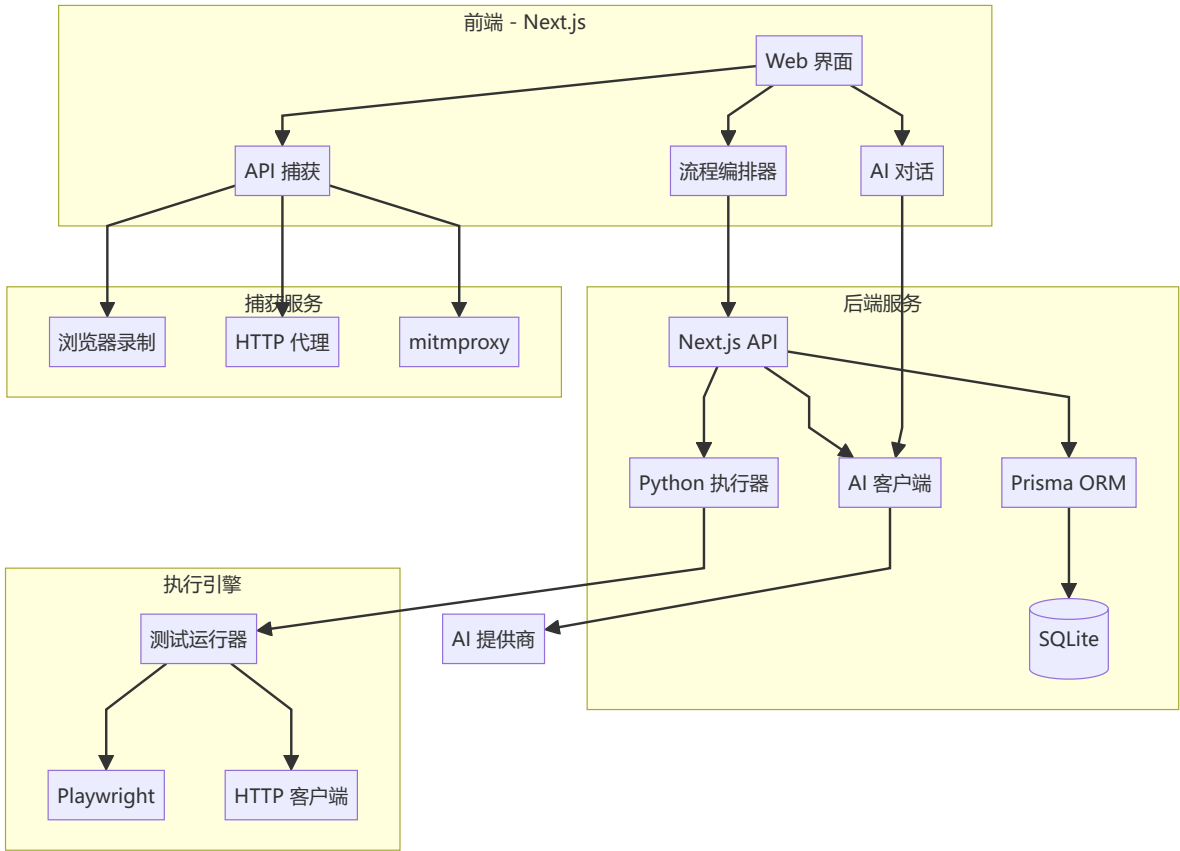
- └─ API 路由: Next.js API Routes
- └─ 数据库: SQLite + Prisma ORM
- └─ 执行引擎: Python (FastAPI)
- └─ 自动化: Playwright
- └─ 捕获服务: mitmproxy

AI 集成:

多提供商支持

- └─ OpenAI (GPT-4, GPT-3.5)
- └─ DeepSeek (深度求索)
- └─ Claude
- └─ 百度文心一言
- └─ 阿里通义千问
- └─ 智谱 AI
- └─ ollama (本地部署)

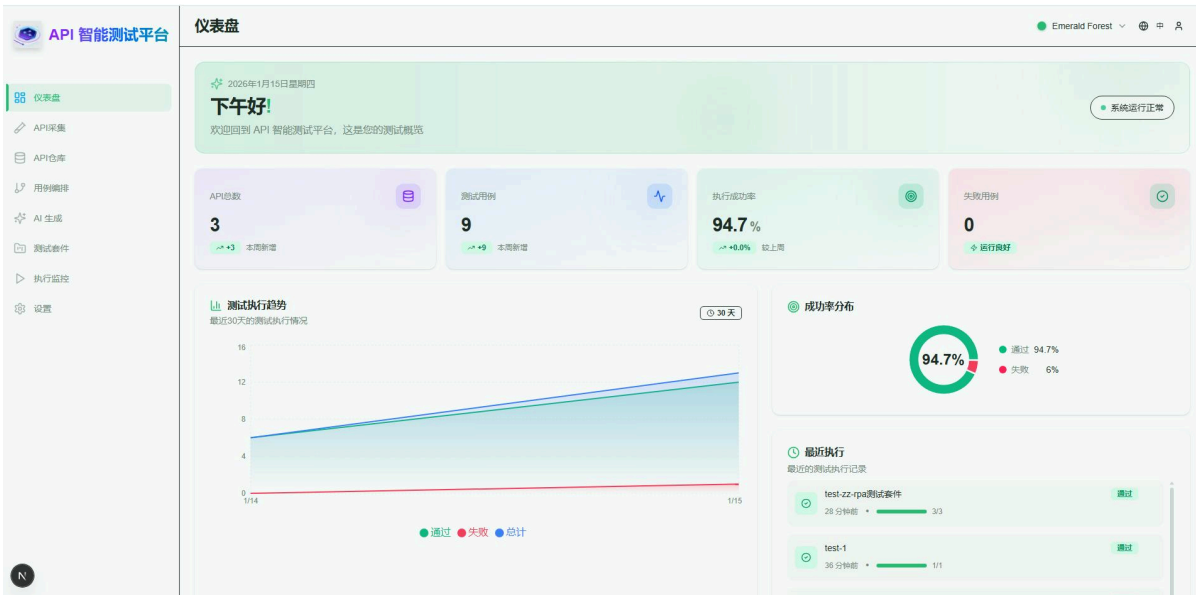
系统架构图



截图与演示

产品截图

仪表盘 - 概览和统计分析



API 仓库 - 管理 API 集合

API 智能测试平台

仪表盘

API 采集

API 仓库

用例编排

AI 生成

测试套件

执行监控

设置

API 仓库

分类

共 3 个 API

全部 API

收藏

搜索 API 名称、路径、描述

全部方法

搜索

GET GET-login_guide-4fba3971ce.js

https://pss.bdstatic.com/static/superman/js/components/login_guide-4fba3971ce.js

分类 test-11 58ms

POST POST-login

http://192.168.7.188:12222/login/eco-system/user/login

分类 test-xx-rpa 用户管理 登录 162ms

POST POST-loginByAesCode

http://192.168.12.64:18088/hkj/camp/loginByAesCode

分类 ITOMV 用户管理 登录 73ms

第 1 页, 共 1 页

上一页 下一页

AI 生成 - 自然语言生成测试

API 智能测试平台

仪表盘

API 采集

API 仓库

用例编排

AI 生成

测试套件

执行监控

设置

AI 生成

新对话

新对话

1月15日 16:19

帮我生成API 路径为/login/eco-system/u...

1月15日 15:53

帮我生成API 路径: /login/eco-system/u...

1月15日 14:56

帮我生成这个接口的hikj/camp/loginByAes...

1月15日 13:45

用户登录接口, API 路径: /login/eco-syste...

1月14日 09:45

用户登录接口, API 路径: /login/eco-syste...

1月13日 13:39

AI 智能生成测试用例

使用自然语言描述测试需求, 提供越详细的 API 信息 (如分类、标签、名称、路径), AI 越容易精准查询并生成测试用例

示例 1: 用户登录接口, API 路径: /api/v1/auth/login, 分类: 用户认证, 验证 username 和 password 必填, 成功返回 token

示例 2: 创建资源接口, API 路径: /api/v1/resources, 标签: 资源管理, 验证必填字段和数据格式, 成功返回资源 ID

示例 3: 查询列表接口, API 名称: 获取用户列表, 路径: /api/v1/users, 分类: 数据查询, 支持分页参数 (page、pageSize), 验证返回数据结构

生成类型

API 测试

E2E 测试

描述接口测试需求, 建议提供详细 API 信息以便精准查询:

- API 分类 (如: 用户认证、资源管理、数据查询等)
- API 标签或名称 (如: 创建用户、获取列表等)
- API 路径 (如: /api/v1/users)

Enter 发送 Shift+Enter 换行

API 智能测试平台

仪表盘

API 采集

API 仓库

用例编排

AI 生成

测试套件

执行监控

设置

AI 生成

新对话

新对话

1月15日 16:19

帮我生成API 路径为/login/eco-system/u...

1月15日 15:53

帮我生成API 路径: /login/eco-system/u...

1月15日 14:56

帮我生成这个接口的hikj/camp/loginByAes...

1月15日 13:45

用户登录接口, API 路径: /login/eco-syste...

1月14日 09:45

用户登录接口, API 路径: /login/eco-syste...

1月13日 13:39

帮我生成API 路径为...的测试用例

15:52

思考过程

13 步

1. 正在分析您的测试需求...

2. 我来帮您生成登录接口的测试用例。首先让我搜索这个API的详细信息。

3. 找到了这个API。现在让我获取它的详细信息, 以便了解参数结构。

4. 根据API详情, 我了解到这是一个登录接口。使用POST方法。请求体包含"username"和"password"字段。响应中包含"accessToken"和"refreshToken"等信息。

5. 这是一个接口测试场景, 我将为此登录接口生成多个测试用例, 包括正常登录和异常情况。

6. 已为您设计 4 个测试用例:

7. 1. 用户登录 - 正常用例 - 1 个步骤 - 无清理

8. 2. 用户登录 - username 为空 - 1 个步骤 - 无清理

9. 3. 用户登录 - password 为空 - 1 个步骤 - 无清理

10. 4. 用户登录 - 错误的用户名密码 - 1 个步骤 - 无清理

11. 分类: 用户管理

12. 标签: ["参数校验", "接口测试", "AI生成"]

13. 现在生成编排指令:

工具调用

3 个工具

3 成功

生成类型

API 测试

E2E 测试

描述接口测试需求, 建议提供详细 API 信息以便精准查询:

- API 分类 (如: 用户认证、资源管理、数据查询等)
- API 标签或名称 (如: 创建用户、获取列表等)
- API 路径 (如: /api/v1/users)

Enter 发送 Shift+Enter 换行

流程编排器 - 可视化测试编排

API 智能测试平台

仪表盘

API采集

API仓库

用例编排

AI生成

测试套件

执行监控

设置

用例编排

← 返回列表

用户登录 - 正常用例

已发布

保存草稿

保存并发布

运行测试

测试用户登录接口, 使用正确的用户名和密码, 验证成功返回Access Token

分类 (可选)

参数编辑

接口测试

AI生成

添加标签...

节点面板

拖拽节点到画布, 连接节点形成流程

API请求
API请求

并行执行
并行执行

延迟
延迟

断言
断言

结束节点
结束节点

开始

POST POST-login
已配置 4 个断言

完成

执行监控 - 实时测试执行

API 智能测试平台

仪表盘

API采集

API仓库

用例编排

AI生成

测试套件

执行监控

设置

执行监控

实时查看测试套件执行状态和结果

清空记录 刷新

test-zz-rpa测试套件 已完成

开始时间: 33分钟前 耗时: 6.87秒 触发方式: 手动

重试 查看日志 查看详情

进度: 100%

3 通过 · 0 失败 · 总计 3 · 通过率 100%

步骤统计: 3 通过 · 0 失败 · 总计 3

test-1 已完成

开始时间: 41分钟前 耗时: 2.55秒 触发方式: 手动

重试 查看日志 查看详情

进度: 100%

1 通过 · 0 失败 · 总计 1 · 通过率 100%

步骤统计: 1 通过 · 0 失败 · 总计 1

test-zz-rpa测试套件 已完成

开始时间: 42分钟前 耗时: 40.56秒 触发方式: 手动

重试 查看日志 查看详情

进度: 100%

2 通过 · 1 失败 · 总计 3 · 通过率 67%

步骤统计: 2 通过 · 1 失败 · 总计 3

电商项目测试 已完成

开始时间: 52分钟前 耗时: 10.05秒 触发方式: 手动

重试 查看日志 查看详情

进度: 100%

3 通过 · 0 失败 · 总计 3 · 通过率 100%

步骤统计: 3 通过 · 0 失败 · 总计 3

执行详情

test-zz-rpa测试套件

成功 查看日志 重试执行

执行时间: 6.87秒
2026/11/5 15:32:43

用例统计: 3/3
通过率 100%

步骤统计: 3/3
失败 0

执行进度: 3/3 已完成

用例执行详情

共 3 个用例

用户登录 - password 为空
步骤: 1/1 耗时: 1.71秒

成功 查看日志 展开

用户登录 - username 为空
步骤: 1/1 耗时: 1.50秒

成功 查看日志 展开

用户登录 - 正常用例
步骤: 1/1 耗时: 1.60秒

成功 查看日志 展开

环境配置快照

执行时使用的环境配置

配置来源: 全局配置

使用场景

✓ 电商测试

测试场景：完整购买流程

- 用户登录
- 浏览商品
- 加入购物车（并发）
 - 商品 A
 - 商品 B

- | └─ 商品 C
- |─ 使用优惠券
- |─ 结算
- |─ 支付
- └─ 清理：删除测试订单 ✨

✅ 微服务测试

测试场景：服务编排

- |─ 认证服务
 - | └─ 提取：accessToken
- |─ 用户服务（并发）
 - | └─ 创建资料
 - | └─ 更新偏好
 - | └─ 上传头像
- |─ 通知服务
 - | └─ 断言：邮件已发送
- └─ 审计服务
 - └─ 验证：所有操作已记录

✅ CI/CD 集成

```
# .github/workflows/api-tests.yml
name: API 测试
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: 运行 API 智能测试平台 测试
        run: |
          docker-compose up -d
          npm run test:api
```

📁 文档与配置

📖 文档中心

文档	说明
Docker 快速开始	Docker 部署指南
完整部署指南	详细部署文档
环境变量说明	配置选项说明
文档中心	完整文档索引

⚠️ 常见问题：执行器连接失败

如果浏览器提示 `Failed to fetch` 或 执行器地址是否正确：`http://localhost:8001:`

原因：`NEXT_PUBLIC_EXECUTOR_URL` 配置不正确

解决方法：

```
# 1. 修改 .env 文件中的执行器地址为服务器 IP
NEXT_PUBLIC_EXECUTOR_URL=http://你的服务器IP:8001

# 2. 重新构建（重要！NEXT_PUBLIC_ 变量在构建时嵌入）
npm run build

# 3. 重启服务
npm run start
```

💡 提示：Docker 部署时需要在构建时传入 `--build-arg`
`NEXT_PUBLIC_EXECUTOR_URL=http://你的服务器IP:8001`
